

# Air-Gapped Local AI Infrastructure Experiment (Exo P2P Mesh)

## Overview & Goal

The objective of this project was to build a 100% offline, air-gapped, decentralized AI inference cluster using everyday, repurposed hardware. By distributing and pooling the memory resources of older, broken, or idle devices, the goal was to create a unified peer-to-peer (P2P) network capable of loading and running frontier-level LLMs—specifically testing the **Gemma 4 12B Unified model**—entirely bypassing reliance on cloud layers or external network handshakes.

---

## Infrastructure & Initial Setup

To establish an isolated, high-speed connection between the devices, a dedicated Linksys Wi-Fi modem was configured to broadcast a local 5G network named `AI_Cluster_5G`.

The hardware pool consisted of the following target nodes:

- **Primary Node:** MacBook Pro (16GB Unified Memory / Apple Silicon Architecture)
  - **Node 1:** MacBook Pro (Intel Core / Non-functional screen)
  - **Node 2:** Samsung Galaxy S21 Ultra (Android 14 / One UI / Partially functional screen)
  - **Node 3:** Samsung Galaxy Note 9 (Android 10 / One UI Legacy / Non-functional screen)
  - **Visual Console:** Samsung Galaxy Tab S6 Lite (Configured to listen on port `52415` to host the P2P cluster mesh monitor network layer)
- 

## Node Configuration & Deployment

### 1. Headless Server Configurations

- **Intel MacBook Pro:** Configured as a headless server running continuously with a closed lid by overriding native macOS kernel energy management flags via the terminal ( `sudo pmset -a disablesleep 1` ). Secure Shell (SSH) was enabled for remote workspace administration.
- **Galaxy Note 9:** Optimized for legacy Android APIs by activating a permanent hardware wake lock ( `termux-wake-lock` ) to keep CPU cores active when the screen dimmed.

Termux was whitelisted as "Unrestricted" under One UI battery management to prevent the background execution engine from terminating active threads.

## 2. Environment & Toolchain Provisioning

The mobile devices were provisioned with sandboxed Linux subsystems natively on the ARM64 architecture using the latest stable F-Droid distributions of Termux.

- Internal repository maps on the legacy Note 9 were rewired using `termux-change-repo` to force connections via stable mirrors, bypassing broken default package indexing.
- Core development dependencies (`git`, `python`, `clang`, and `nodejs-lts`) were compiled and deployed natively across the hardware pool.
- The native Termux-compiled version of the `uv` dependency manager was utilized to bypass standard Linux binary compatibility errors on Android.

## 3. Framework Assembly

The core `exo` repository was initially cloned onto the Galaxy S21 Ultra. After successfully running `npm install` and `npm run build` to compile the visual Node.js dashboard interface natively on the phone, the deployment files were transferred over the local `AI_Cluster_5G` network via Termux Linux to provision the remaining nodes in the array.

---

## Technical Engineering Challenges

During deployment, the cluster encountered several strict cross-platform compatibility and hardware constraints:

- **macOS Dependency Blockers:** The primary node faced initial build crashes due to `nodejs-wheel-binaries` throwing nested CMake build errors through development packages (`basedpyright`). This required bypassing the standard development workflow and enforcing strict runtime-only synchronization (`uv sync --no-dev`).
- **Platform Lockfile Constraints:** Non-standard platform tagging inside Termux caused strict `uv.lock` validation errors on the mobile devices. This was bypassed by isolating clean `uv venv` sandboxes and utilizing direct editable installations (`uv pip install -e .`).
- **Compilation & Compilation Bottlenecks:** Native source-compilation failures occurred on mobile when attempting to build `psutil` and the core Rust-based `exo-rs` cross-bindings, stemming from hardcoded Android kernel checks and interrupted cargo network fetch loops. Non-critical metrics dependencies had to be surgically stripped from `pyproject.toml`.
- **Thermal Panic Limits:** During intensive cryptographic and optimization tasks, the legacy Note 9 node triggered severe hardware thermal panic limits, causing aggressive

background process throttling and OS-level termination of the Termux server threads.

---

## Final Architectural Conclusion

While this experiment successfully validated the capabilities of local peer-to-peer P2P UI discovery and network tensor mapping, the project ultimately hit an insurmountable wall due to **hardware incompatibility and heterogeneous edge-computing constraints**.

Attempting to split and compute unified model layers across vastly different hardware architectures (Apple Silicon MLX, Intel x86, and mobile ARM64 kernels) created terminal bottlenecks. The extreme resource limitations and thermal throttling of the legacy mobile hardware, combined with the structural friction of cross-compiling core Rust bindings natively for Android, proved that building a stable, high-performance distributed runtime across this specific pool of mixed hardware is not possible at this moment.

For those interested in exploring decentralized local AI orchestration or contributing to peer-to-peer ecosystem modeling, visit the official framework repository:

👉 For more information, check out [Exo on GitHub](#).